

第六章：语义分析

抽象地址
符号表
符号表管理实例

1. 抽象地址

- ◆ 1.1 存储空间分配
- ◆ 1.2 层数定义
- ◆ 1.3 抽象地址结构
- ◆ 1.4 空间分配的原则

1.1 存储空间分配

- ◆ 静态分配

在编译时间即为所有数据对象分配固定的地址单元，且这些地址在运行时间始终保持不变。是一种直观的方法，但不支持需要动态申请空间的数据（如指针）。

- ◆ 动态分配

在编译时给程序中变量分配的地址不是真正的地址，而是一个抽象地址，当程序运行时再根据抽象地址分配具体的物理地址。

过程活动记录

- ◆ 每次函数被调用时都会给函数分配一片空间，用以存储如图信息，我们把这片存储空间称作过程活动记录 (AR)
- ◆ 偏移是针对过程活动记录的，通过起始位置+偏移量就可以找到对应的物理位置。
- ◆ 过程活动记录中存储的顺序实际上是处理的先后顺序。

临时变量区

局部变量区

形参区

管理信息

AR

1.2 层数的定义

- ◆ 嵌套式语言中 (典型代表Pascal) :
 - ❖ 主程序设为0层
 - ❖ 主程序直接定义的函数和过程定义为1层
 - ❖ 若某函数为L层, 则该函数直接定义的函数为L+1层
- ◆ 并列式语言中 (典型代表C) :
 - ❖ 全局变量定义为0层, 所有函数为0层, 函数中定义的变量为1层

1.3 抽象地址的结构

层数	偏移
----	----

- ◆ 抽象地址的形式是一个二元组，由层数和偏移组成
- ❖ 层数主要是针对嵌套式语言，表示的是某个函数所处的嵌套定义层数
- ❖ 偏移是针对过程活动记录的一个相对偏移量

1.4 空间分配原则

- 临时变量分配一个单元(这个只是假设, 具体情况根据目标机和操作数的类型决定)
- 局部变量按类型大小分配单元
- 形参
 - 地址引用型形参: 分一个单元
 - 值引用型形参: 按类型大小分
 - 过/函形参: 分两个单元(入口地址, Display表信息)

2. 符号表概述

- ◆ 符号表在编译的不同阶段都要用到
 - 在语义分析阶段，符号表所登记的内容将用于语义检查（如检查一个名字的使用和先前的声明是否一致）；
 - 在中间代码生成阶段，用于类型检查及强制类型转换；
 - 在目标代码生成阶段，当对名字进行地址分配时，符号表是地址分配的依据（抽象地址转换成物理地址）。

- ◆ 合理地组织符号表的意义：

在编译程序工作的过程中，需要对符号表进行频繁的访问（查表和填表等操作），因此合理地**组织符号表**，并相应地选择好查表和填表的方法，是提高编译程序效率的重要一环。

组织符号表主要解决的问题：

符号表的总体组织（单表/多表）

符号表表项的排列（线性表/散列表/有序表）

符号表的局部化（局部符号表/全局符号表）

2.1 符号表的总体组织

- ◆ 符号表存储的是标识符的语义信息，包含两部分：**标识符**的名字；**语义字**（种类、类型、抽象地址...）
- ◆ 不同类别的标识符所含的信息（语义字）是不同的。
- ◆ 符号表的总体组织既可以采用多表结构，也可以采用单表结构，也可以二者折中。究竟采用哪种结构并没有统一的规定，编译程序可根据实际处理语言的需要进行选择。

(1)、多表结构

- ◆ 是将属性完全相同的那些符号存放在一张符号表中，从而有常量表、类型表、变量表等。
- ◆ 多表结构的优点：每个符号表的属性个数和结构完全相同，各个表项是等长的，并且表项中的每个属性都是有效的，管理起来方便一致，空间效率高。
- ◆ 多表结构的缺点：编译程序将同时管理若干个符号表，增加了总体管理的工作量和复杂度。并且对各类标识符的共同属性如类型的管理必须设置重复的运行机制。

(2)、单表结构

是将所有符号都组织在一张符号表中。

- ◆ 单表结构的优点：总体管理集中单一，且不同种类符号的共同属性可一致地管理和处理。
- ◆ 单表结构的缺点：由于符号属性的不同，为完整表达各类符号的全部属性必将出现不等长的表项，以及表项中属性位置交错重叠的复杂情况，极大地增加了符号表管理的复杂度。
- ◆ 可行的解决方法：把所有符号的可能属性作为符号表表项属性。
 - 优点：有助于降低符号表管理的复杂度；
 - 缺点：对于某些类具体符号可能增加无用的属性空间，从而增加了空间开销。

(3)、折中方式

根据符号属性相似程度分类组织成若干张表，每张表中记录的符号都有比较多的相同属性。

按折中方式重新组织上例中的3类符号，可构成2张符号表，如图所示：

符号	属性值1	属性值2	属性值3	属性值4

第一，二类
符号之符号表

符号	属性值2	属性值5	属性值6

三类符号符号表

2.2 符号表表项的排列

- ◆ 研究符号表项排列的意义：编译过程中，每当遇到一个名字都要查找符号表。如果发现一个新名字，或者发现已有名字的新信息，则要修改符号表，填入新名字和新信息。符号表被频繁地用来建表、查表、填充和引用表的属性，因此表项的排列组织对系统运行的效率起着十分重要的作用。
- ◆ 由于符号表的建立是一次性的，符号表的查找是重复性的，因此查表的方法更为重要，对查表效率的要求决定了建表的方法。

(1)、线性组织

- ◆ **录入方式** 符号表中表项按符号被扫描到的先后顺序录入；
- ◆ **线性组织的优点** 没有空白项，存储空间效率高。
- ◆ **线性组织的缺点** 运行效率低，特别当表项数目较大后效率就非常低。对于符号个数无法确定的情况下，无法确定符号表的总长度。但当事先能确定符号个数且个数不大（公认小于20）的情况下，使用线性组织是非常合适的。

(2)、有序组织

有序组织的符号表，就是在符号表中的表项按其符号的字符代码串（可以看成是一个整数值）的值的大小从大到小（或从小到大）排列的。

关于有序表的表项建立及符号查找，通常采用“二分法”

(3)、散列组织

- ❖ 一个符号在散列表中的位置，由对该符号代码值进行某种函数操作（杂凑函数）所得到的函数值来确定的
- ❖ 假设选定杂凑函数fhash，对符号代码值杂凑运算之后得到杂凑值是Vhash，可表示为：

$$Vhash = fhash(\langle \text{符号代码值} \rangle)$$

- ❖ 设符号的散列位置Lhash：

$$Lhash = \text{mod} (Vhash, N)$$

其中N为散列表的表长

优缺点：效率最高，但实现上较复杂而且要消耗一些额外的存储空间。

2.3 符号表查表技术

- ◆ 顺序查表法
- ◆ 折半查表法（二分法）
- ◆ 散列查表法（哈希表）

3. 符号表的局部化处理

- ◆ 源程序的局部化单位
- ◆ 标识符的作用域
- ◆ 局部化区的语义错误检查
- ◆ 标识符的处理原则
- ◆ 符号表的局部化组织方法

3.1 源程序的局部化单位

程序中允许有声明的部分称为一个局部化单位,通常是一个子程序(函数、过程)或分程序:

```
Procedure P (.....) begin.....end
```

```
Function F (.....) begin..... end
```

```
int G (.....) {.....}  
{.....}
```

- 注: 1. Pascal语言中主程序也可以被看作为一个局部化单位;
2. 局部化单位可以嵌套, 不能交叉。

Pascal过程声明嵌套的例子

1. procedure P();

2. var x,y:integer;

3. **procedure Q();**

4. **var x,z:real;**

5. **begin**

6. x..z..y..

7. **end**

8. **begin**

9. x.....y..

10. **end**



局部化单位₂



局部化单位₁

带有分程序的C程序的例子

```
int main( )
{
    int a = 1;
    int b = 1;
    {
        int b = 2;
        {
            int a = 3;
            printf("a = %d, b = %d\n",a, b);
        }
        {
            int b = 4;
            printf("a = %d, b = %d\n",a, b);
        }
    }
    printf("a = %d, b = %d\n", a, b);
}
```

局部化单位1

局部化单位2

局部化单位3

局部化单位4

a = 3, b = 2

a = 1, b = 4

a = 1, b = 1

3.2 标识符的作用域

- ◆ 一个局部化单位所建立的所有符号表表项称为该局部化单位的符号表。（局部化单位符号表中表项的存储位置不一定相邻）
- ◆ 一个局部化单位的符号表的有效范围是该局部化单位。
- ◆ 在程序的点P处，对它有效的符号表是包含P的那些嵌套外层的局部化单位的符号表。

符号表局部化处理的本质

语义分析时，在程序的点P处，遇使用性出现的标识符，查符号表只能按着由内层到外层的次序（体现标识符的可视性作用域）查在此处有效的那些局部化单位的符号表，而不访问在此无效的那些符号表，这就要求编译程序合理组织符号表，使其能够在程序的每点判断出哪些局部化单位的符号表是有效的，这就是符号表局部化处理的本质。

3.3 局部化区的语义错误检查

- ◆ 变量的重复声明

在一个程序的局部化区里，同一个标识符不能被声明两次。

- ◆ 标识符的使用有无声明

强类型语言规定标识符必须先声明后使用，弱类型语言无要求。

3.4 标识符处理原则

- ❖ 遇到声明性标识符时，首先查找当前局部化单位的符号表中有无与此同名者，若有，则报告重复定义的语义错误；若无，则将该标识符的名字及其属性填入符号表。
- ❖ 遇到使用性出现，按由内到外的次序依次查其嵌套外层的局部化单位的符号表，若有，则根据查得的属性判断对其使用是否合法；若无，则报告有使用而无声明的语义错误。

例1.(注:将fun1和fun2作为无参函数) 全局符号表

```
1.  int x=10;
2.  fun1( )
3.  {int x=20, z;
4.      ....
5.      z=x+10;
6.      ....
7.  }
8.  fun2( )
9.  {int y:
10.     y=x+30;
11. }
```

x	intPtr	varKind	dir	0	m	10
fun1		routKind	...			
fun2		routKind	...	..		

fun1的符号表

x	intPtr	varKind	dir	1	m	20
z	intPtr	varKind	dir	1	m+1	

fun2的符号表

y	intPtr	varKind	dir	1	m	
---	--------	---------	-----	---	---	--

例2： (建立符号表表项时检查有无重复定义)

全局符号表

.....

```
1.  int x=10;
2.  fun1( )
3.  {
4.      int x=20, z;
5.      int x=30;
6.      ....
7.      z=x+10;
8.      ....
9.  }
10. fun2( )
11. {int x=30;
12.     y=x+100;
13. }
```

x	intPtr	varKind	dir	0	m	10
fun1		routKind	...			
fun2		routKind	...	..		

fun1的符号表

x	intPtr	varKind	dir	1	m	20
z	intPtr	varKind	dir	1	m+1	

fun2的符号表

x	intPtr	varKind	dir	1	m	30
---	--------	---------	-----	---	---	----

3.5 符号表的局部化组织方法

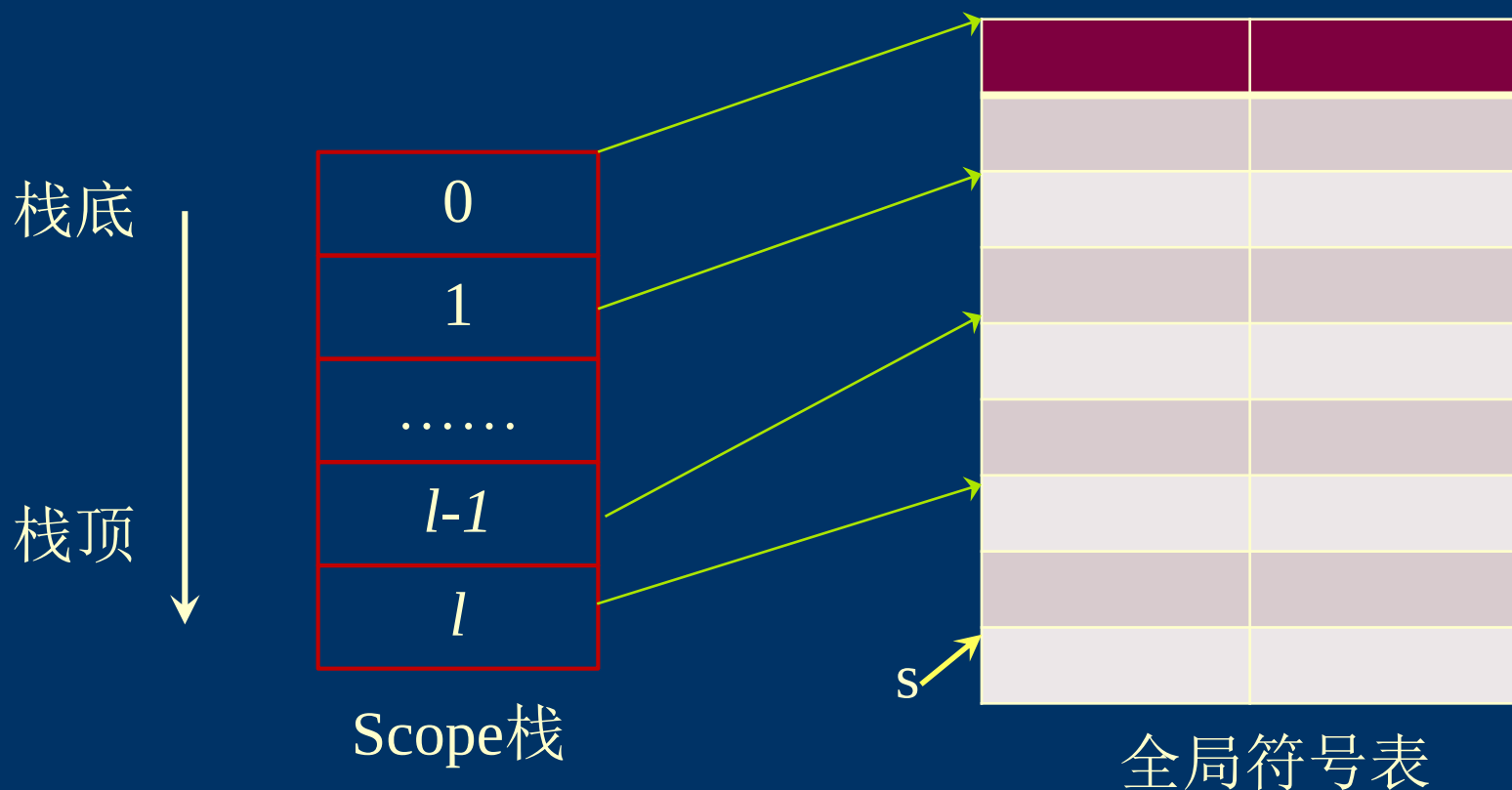
从符号表的组织方式上看，可分为两大类：一类是局部式，一类是全局式。

- **全局式符号表**是把整个程序的符号表作为一个表，即建表和查表单位是整个符号表；
- **局部式符号表**是指把每个局部化单位的符号表作为一个独立的表来处理，即把每个局部符号表作为建表和查表单位。

(一)、全局式线性组织的符号表的局部化处理思想

- 每当进入一个局部化区，记住本局部化区符号表的始地址；
- 每当遇到定义性标识符时，构造其语义信息并查本本局部化区符号表，若查到同名标识符，则表示有错，否则往符号表里填写标识符及其语义信息。
- 每当遇到使用性标识符时，查整个符号表（从后往前），若查不到同名标识符，则表示有错（因为我们假定声明在前），否则找出相应语义信息并将它传给有关部分。
- 每当结束一个局部化区时，“删除”本局部化区符号表。

- 用一个Scope栈来实现标识符的嵌套作用域(栈便于体现标识符的嵌套作用域原则)。
- $\text{Scope}[i]$ 指向 i 层符号表始地址, l 表示层数计数器, s 表示符号表的最后一项的地址

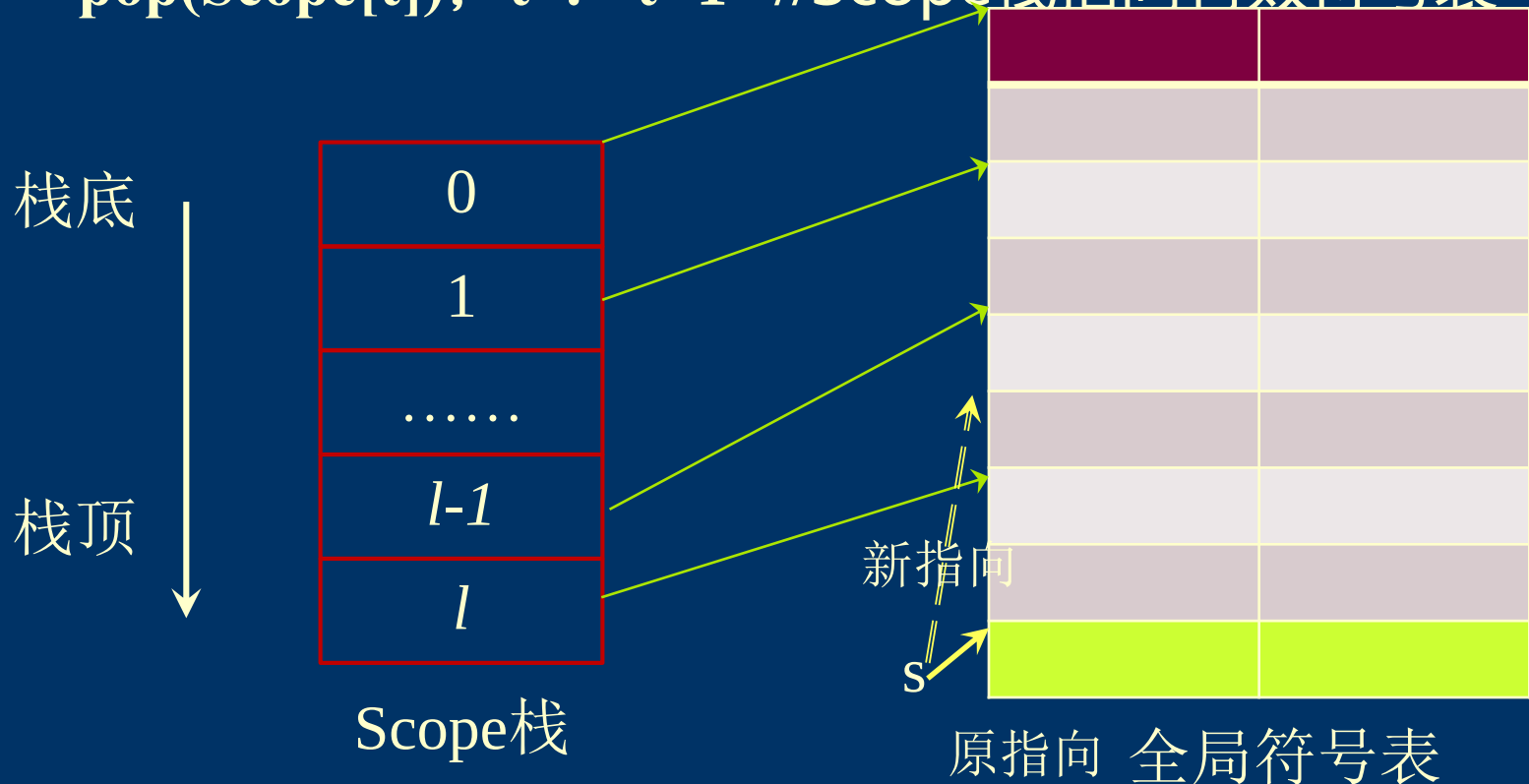


➤ 当退出一个局部化单位时，该局部化单位中声明的标识符不再有效，应删除掉该局部化单位的符号表。“删除”本层符号表的方法一般有两种：

1. 真删除法：依次 $\text{pop}(\text{SymbTab}[s])$ ， $s := s - 1$ ，直至

$s := \text{Scope}[\ell] - 1$ // s 指向有效符号表的最后一项

$\text{pop}(\text{Scope}[\ell])$ ； $\ell := \ell - 1$ // Scope 栈指向有效符号表



2. 驻留法: 不删除表, 但采取一定措施不去查那些已无效的符号表部分。办法是: 每当退出一个局部化区时, 将一个跳转项添加到符号表中, 使得该局部化区包含的表项不被访问到:

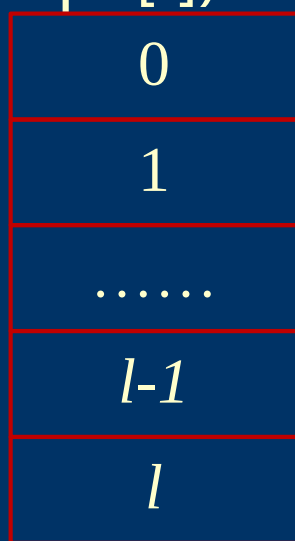
$s := s + 1 ;$

$\text{SymbTab}[s] := (\# , \text{Scope}[\ell] - 1)$

$\text{pop}(\text{Scope}[\ell])$

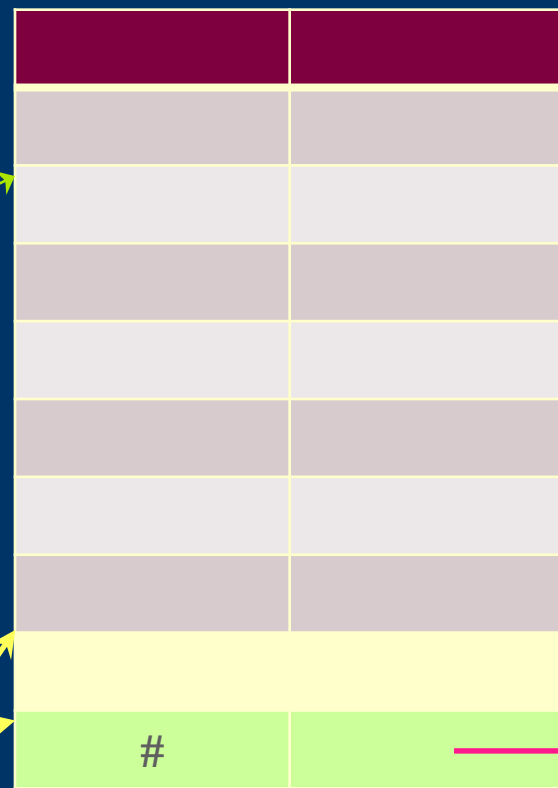
栈底

栈顶



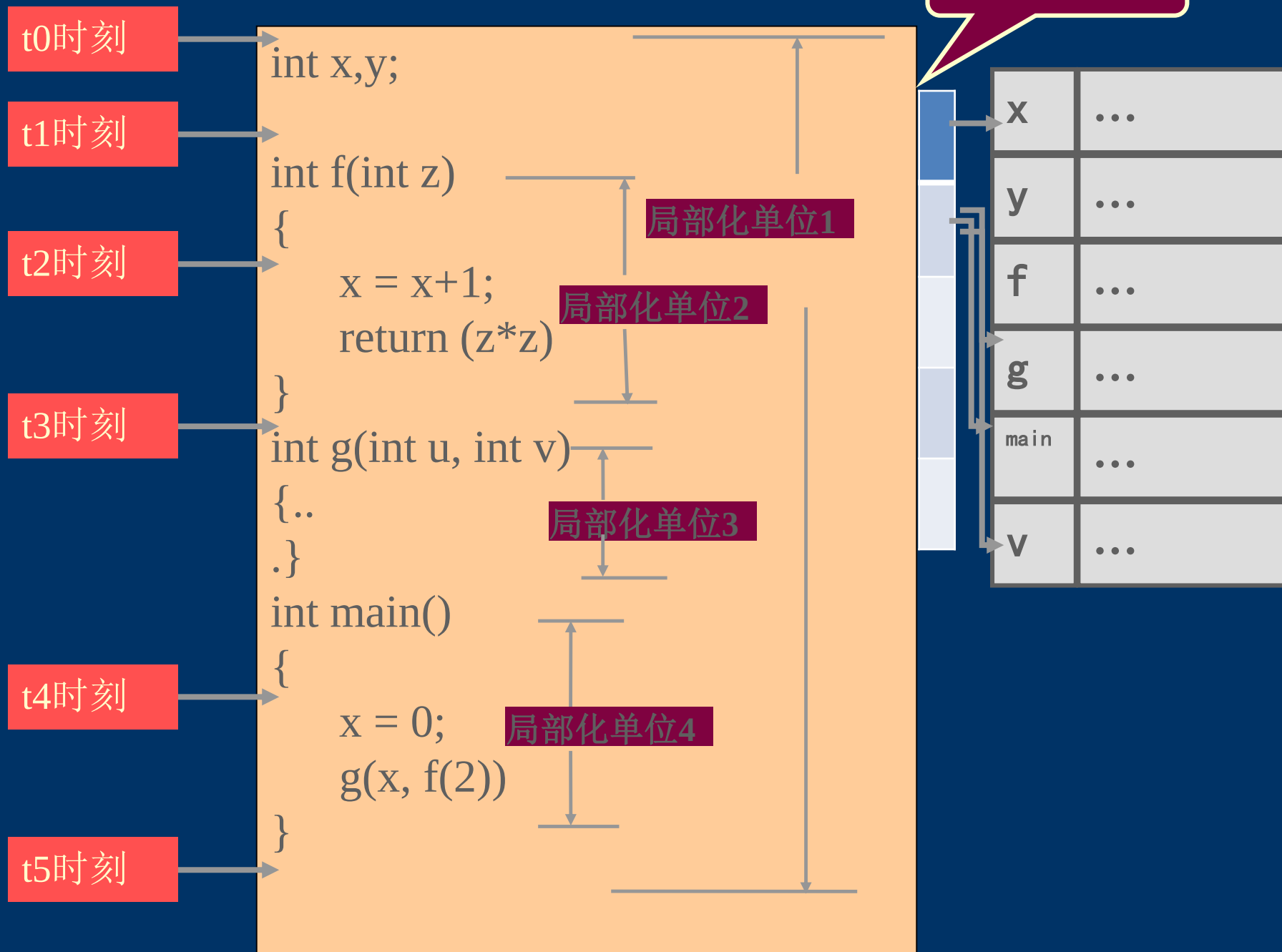
Scope栈

原指向
s
新指向

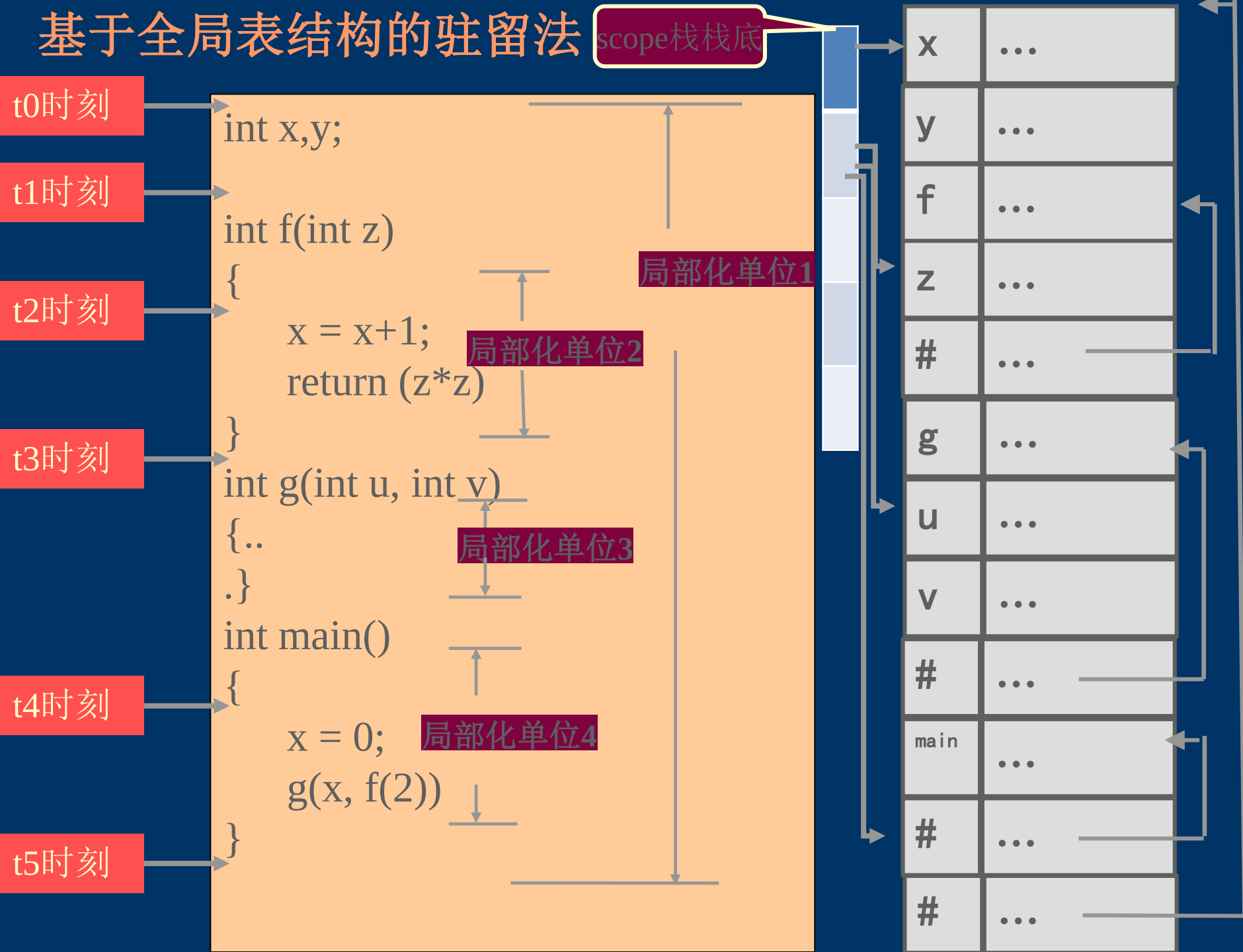


全局符号表

基于全局表结构的真删除法



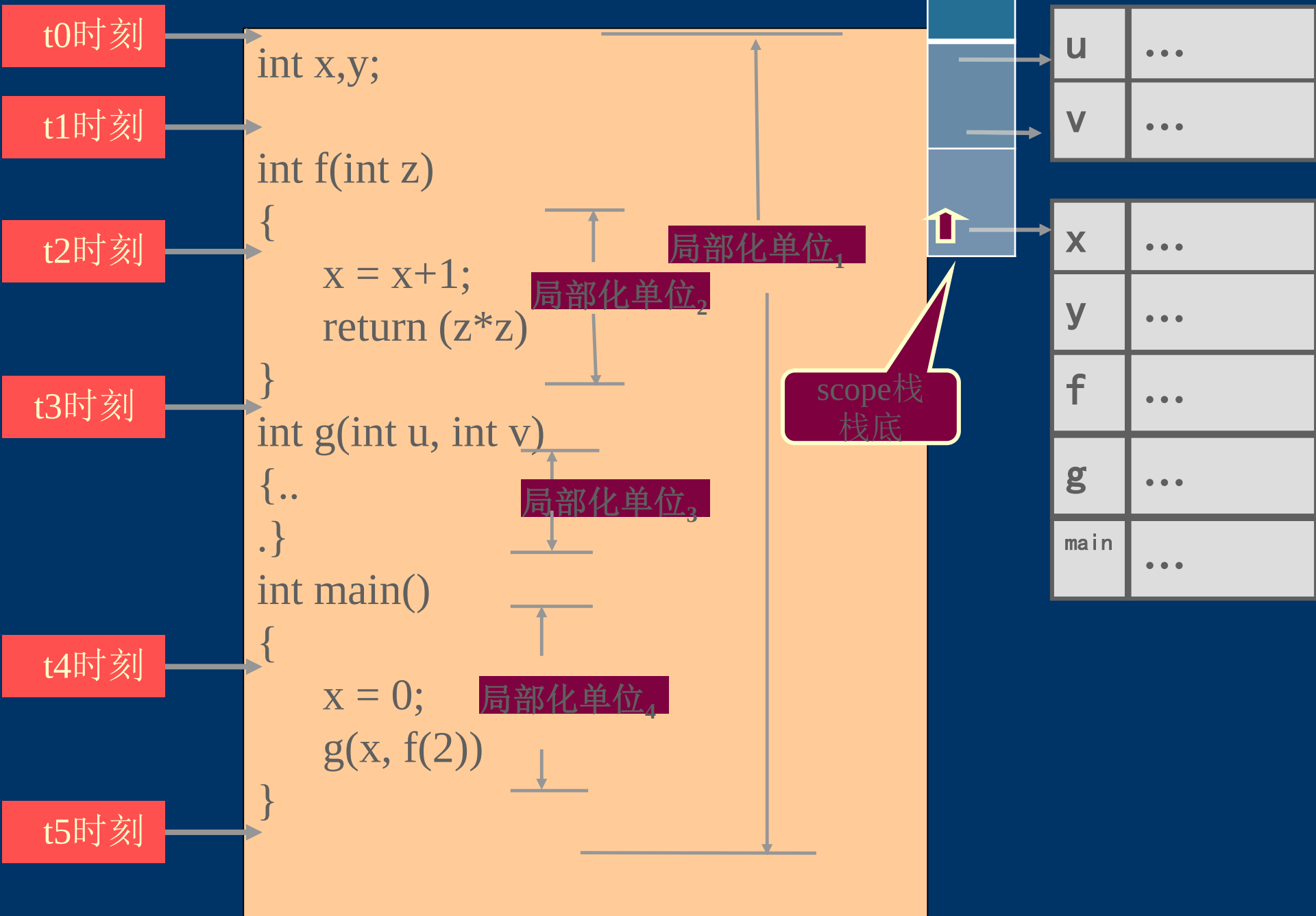
基于全局表结构的驻留法



（二）、局部式线性组织的符号表的局部化处理思想

- ◆ 在不同时刻，不是所有的局部符号表都有效。为了标记在某一时刻，哪些符号表是有效的，用到一个辅助数据结构Scope栈。
- ◆ Scope栈用于存放当前有效的局部化单位的局部符号表的始地址。每当进入一个新的局部化区，就将它的符号表始地址存入Scope栈，当退出一个局部化区时，再将Scope栈的栈顶元素弹出。
- ◆ 删除法、驻留法之分都是基于全局表的结构，而对局部表只有删除法。

局部表结构的管理



4.符号表管理实例

- ◆ 4.1 Pascal语言符号表
- ◆ 4.2 C语言符号表
- ◆ 4.3 嵌套式语言和并列式语言的比较

4. 1. Pascal语言的符号表的管理

- ◆ Pascal语言的最大特点就是允许嵌套的过程声明。规定Pascal主程序的层数为0，在主程序中声明的标识符(包括过/函标识符)的层数为1，在第i层过/函中声明的标识符(包括形参和局部标识符)的层数为 $i+1$ ($i \geq 1$)。
- ◆ 删除法实现符号表的局部化
- ◆ 驻留法实现符号表的局部化

●删除法---Pascal语言符号表的实例

```
PROGRAM ep;
```

```
procedure P1( )
```

```
    var i,j,k:integer; {t1}
```

```
    procedure Q2( )
```

```
        var a, b: real;
```

```
        procedure R3( ) {t2}
```

```
            var a,b:boolean; {t3}
```

```
            begin
```

```
                R的过程体 (a, b, i)
```

```
            end
```

```
        begin
```

```
            Q的过程体 (a, b, i)
```

```
        end;
```

```
        procedure S2( )
```

```
            var c,e:char; {t4}
```

```
            begin
```

```
                S的过程体 (a, b, i, Q)
```

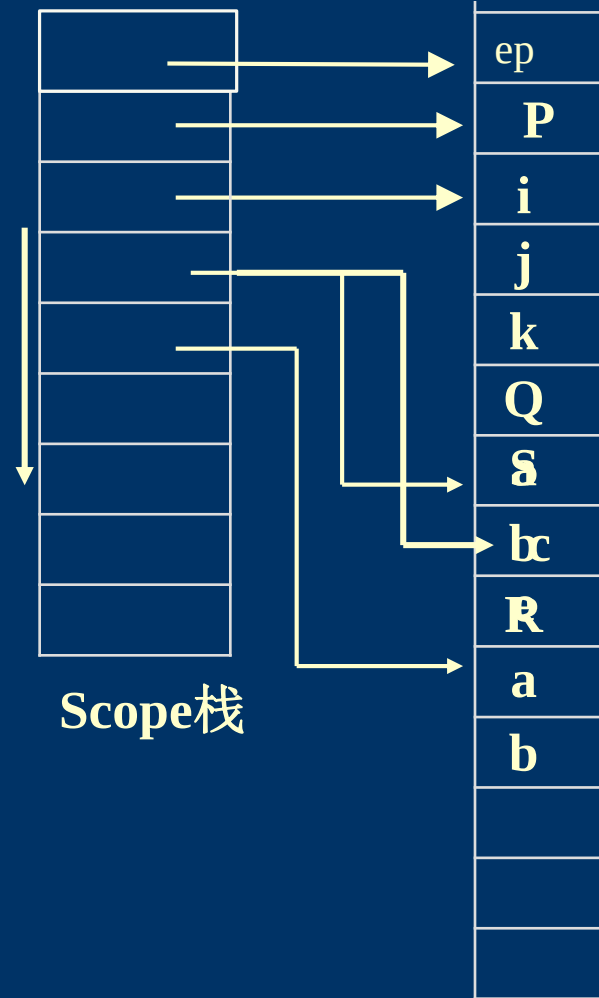
```
            end ; {t5}
```

```
    begin
```

```
        P的过程体 (a, b, i, P, Q, S)
```

```
    end;
```

```
begin.. P( ) ; .. end.
```



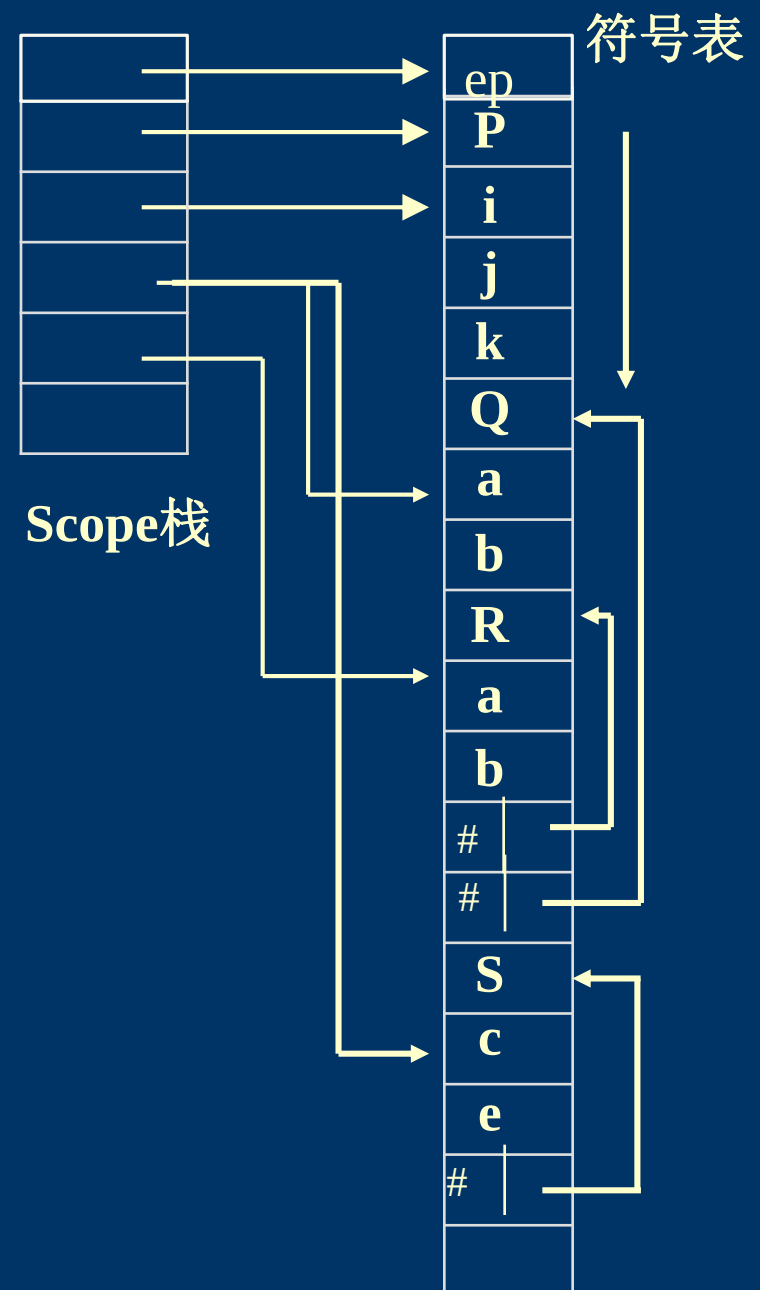
- ◆ 驻留式方法又可分为标记法和跳转法。
- ◆ 标记法是给每个表项增加一个标记域，当某个表项变为无效时，就将该标记域设为无效；
- ◆ 跳转法的主要思想是（考虑从后往前的情形）：
- 每当退出一个局部化区时，将一个跳转项添入到符号表中：
 $s := s + 1$; $\text{SymbTab}[s] := (\#, \text{Scope}(l) - 1)$; $\text{pop}(\text{Scope}[l])$;
- 当查表时，从当前符号表的末项 $\text{SymbTab}[s]$ 开始往前查，当遇到跳转项时跳到跳转项的地址部分所指向的地址继续查找。

例如：如果要查的是X的属性，则查表过程大致可描述如下：（开始i 取s 值）

```
[1] if SymbTab[i].name = X then
      return(SymbTab[i].attrib);
[2] if SymbTab[i].name = #
      then i = SymbTab[i].addr
      else i = i - 1;
[3] if i = 0 then Error else goto [1].
```

● 跳转法——Pascal语言符号表的实例

```
PROGRAM ep;  
1. procedure P( )  
2.     var i, j, k: integer; {t1}  
3.     procedure Q( )  
4.         var a, b: real;  
5.         procedure R( ) {t2}  
6.             var a, b: boolean; {t3}  
7.             begin  
8.                 R的过程体 (a, b, i)  
9.             end;  
10.        begin  
11.            Q的过程体 (a, b, i)  
12.        end;  
13.        procedure S( )  
14.            var c, e: char; {t4}  
15.            begin  
16.                S的过程体 (a, b, i)  
17.            end; {t5}  
18.    begin  
19.        P的过程体 (a, b, i)  
        end  
begin      .....      end.
```



4.2 C语言的符号表的管理

- ◆ C语言不允许函数的嵌套声明。
- ◆ 一个C程序是由若干个函数并列组成的（其中一定要有main函数），这些函数的地位都是相同的。从main函数调用开始执行，在后面声明的函数可以调用在它之前声明的函数。
- ◆ C语言的每个标识符的层次只有两个，即0层和1层。全局标识符和所有的函数标识符的层数是0，函数的形式参数和局部标识符的层数是1。

(1) 简单的C语言符号表_驻留法

```

int x,y;
void swap(int* a, int*b )
{
    int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}
void main()
{
    x = 10;
    y = 20;
    swap(&x,&y);
}
    
```

x	varKind	intPtr	dir	0	0		
y	varKind	intPtr	dir	0	1		
swap	routKind	NULL	actual	0		size	false
a	varKind	atptr	indir	1	off _o		
b	varKind	btptr	indir	1	off _o +1		
tmp	varKind	intPtr	dir	1	off _o +2		
#							
main	routKind	NULL	actual	0	null	size	false
#							
#							

atptr→	1	pointerTy	intPtr
btptr→	1	pointerTy	intPtr

(2) 带有分程序结构的C语言符号表

- 当考虑C语言的分程序语句时，C程序符号表的构造将变得复杂一些。C语言的分程序语句(也称程序块)是由“{”和“}”括起来的程序段，其中可以包含标识符声明，还允许一个块嵌套在另一个块内。
- 一个分程序就是一个局部化区。分界符“{”和“}”标记分程序的开始和结束，分界符保证分程序不是相互独立就是一个嵌在另一个里面。

带有分程序的C语言全局符号表_驻留法

(假定off (偏移) 为0)

```
1.  int main()  
2.  {1  
3.      int a;  
4.      float b, d;  
5.      {2  
6.          int c;  
7.          float a;  
8.          {3  
9.              int d;  
10.             float c;  
11.             {4  
12.                 float d;  
13.                 .....  
14.                 a=b+c+d;  
15.             }4  
16.         }3  
17.     }5  
18.     char d;  
19.     }5  
20. }2  
21. }
```

main	routKind	NULL	actual	0	NULL	false
a	varKind	intPtr	dir	1	0	
b	varKind	realPtr	dir	1	1	
d	varKind	realPtr	dir	1	3	
c	varKind	intPtr	dir	1	5	
a	varKind	realPtr	dir	1	6	
d	varKind	intPtr	dir	1	8	
c	varKind	realPtr	dir	1	9	
d	varKind	realPtr	dir	1	11	
#						
#						
d	varKind	charPtr	dir	1	13 (8)	
#						
#						
#						



带有分程序的C语言全局符号表_删除法

```
int main( )
```

```
{
```

```
int a = 1;
```

```
int b = 1;
```

```
{
```

```
int b = 2;
```

```
{
```

```
int a = 3;
```

```
printf("a = %d, b = %d\n",a, b);
```

```
}
```

```
{
```

```
int b = 4;
```

```
printf("a = %d, b = %d\n",a, b);
```

```
}
```

```
}
```

```
printf("a = %d, b = %d\n", a, b);
```

```
}
```

分程序1

分程序2

分程序3

分程序4

a	1
b	 1
b	 2
b	 4

a = 3, b = 2

a = 1, b = 4

a = 1, b = 1

4.3 嵌套式语言并列式语言的比较

- ◆ 特殊情形，从程序设计语言的角度来说有嵌套式语言和并列式语言
- ◆ 从处理难度角度来说，可能嵌套式语言复杂，并列式简单。嵌套式语言里还要考虑变量运行环境。
- ◆ 并列式语言的层数分成两层，全局量定义成0层，局部量定义成1层。符号表组织的特殊情形是分程序结构，分程序可以是嵌套的，每个分程序看成一个独立的局部化区，但这些分程序中变量的层数是相同的。

语义分析小结

- ◆ 何时建表
- ◆ 每当进入一个新的局部化区建立一个新表
 - 局部符号表，是新建一个表
 - 全局符号表，是新增一个表项
 - 用Scope栈记录每个局部化区符号表的起始地址
 - 用s记录最新表项的地址
- ◆ 何时查表
- ◆ 遇到标识符声明时：检查本局部化区有没有重复声明
- ◆ 遇到标识符使用时：检查标识符有没有声明，使用与声明是否一致？由内而外进行：先查本局部化区符号表，再查外层、次外层...

全局符号表—删除法

- ◆ 整个程序一个符号表，即所有局部化区的符号表都放在一起，所以要用scope栈来记录每个局部化区符号表的起始地址；
- ◆ 标识符及其属性，依其在源程序中出现的顺序依次填入符号表中，填入时要检查是否有重复声明（scope栈栈顶元素指向的表项开始向后（表尾方向）查）；因为局部化区可能嵌套，所以同一局部化区的标识符表项不一定相邻；
- ◆ 每当遇到一个标识符的使用，从s开始向前（表头方向）查，查到的第一个同名标识符即是。若直至表头仍未查到，则为标识符未声明错误；
- ◆ 每当退出一个局部化区，该局部化区所有表项作废，从s开始依次删除，直至删除到scope栈栈顶指针指向的符号表项，将其删除后，将scope栈栈顶元素弹出。

全局符号表—驻留法

- ◆ 整个程序一个符号表，即所有局部化区的符号表都放在一起，所以要用scope栈来记录每个局部化区符号表的起始地址；
- ◆ 标识符及其属性，依其在源程序中出现的顺序依次填入符号表中，填入时要检查是否有重复声明（scope栈栈顶元素指向的表项开始向后（表尾方向）查）；因为局部化区可能嵌套，所以同一局部化区的标识符表项不一定相邻；
- ◆ 每当遇到一个标识符的使用，从s开始向前（表头方向）查，查到的第一个同名标识符即是。若直至表头仍未查到，则为标识符未声明错误；
- ◆ 每当退出一个局部化区，该局部化区所有表项作废，添加一个标记项(#,ptr), ptr是一个指针，指向当前scope栈栈顶元素指向的符号表项的前一项(表头方向)，s指向新增的标记项，将scope栈栈顶元素弹出。

局部符号表

- ◆ 每个局部化单位一张符号表，存放本局部化区的所有标识符及其属性；
- ◆ 每当进入一个局部化区，建立一个新的符号表，将其起始地址存入scope栈；
- ◆ 每当遇到一个标识符的声明，从scope栈栈顶元素指向的符号表的各个表项开始查，是否有重名标识符；
- ◆ 每当遇到一个标识符的使用，首先从scope栈栈顶元素指向的符号表开始查，若查到，则读出其属性进行语义检查；若未查到，则接着从scope栈次栈顶元素指向的符号表查，依次类推，若直至scope栈栈底元素指向的符号表仍未找到该标识符，则说明是标识符未声明错误；
- ◆ 每当遇到一个局部化区的结束，则从scope栈栈顶元素指向的表项开始，依次删除本局部符号表的各个表项，最后弹出scope栈栈顶元素。

C语言和Pascal语言符号表的比较

- ◆ C语言是并列式语言的代表
- ◆ 程序主体由一些并列的函数定义构成
- ◆ “主程序”、每个函数、每个分程序是一个局部化区
- ◆ 全局变量、函数标识符的层数是0，局部变量层数是1
- ◆ 分程序的嵌套不影响变量的层数！！！！
- ◆ 同层变量标识符偏移量，一般从0开始记，依据出现顺序，类型（种类）等进行计算；
- ◆ Pascal语言是嵌套式语言的代表
- ◆ 程序主体由一些嵌套的过/函定义构成
- ◆ 主程序、每个函数是一个局部化区
- ◆ 主程序标识符层数是0，主程序中声明的标识符层数是1，L层的函数中声明的标识符层数是L+1